MODEL PROSES PERANGKAT LUNAK

Software Development Life Cycle (SDLC)

Dosen: Manorang Sihotang,M.Kom,Ph.D

Tujuan Pembelajaran

Memahami konsep Software Development Life Cycle (SDLC).

Menjelaskan karakteristik berbagai model proses perangkat lunak.

Membandingkan kelebihan dan kekurangan setiap model.

Menentukan model proses yang sesuai dengan karakteristik proyek.

Memahami evolusi model proses menuju pendekatan modern.

Konsep Software Development Life Cycle (SDLC)

Pengertian SDLC: kerangka kerja yang menggambarkan tahapan sistematis dalam proses pengembangan perangkat lunak, mulai dari identifikasi kebutuhan hingga perangkat lunak dipelihara atau dihentikan penggunaannya.

Tujuan SDLC:

- Menghasilkan perangkat lunak berkualitas.
- Mengendalikan waktu dan biaya pengembangan.
- Mengurangi risiko kegagalan proyek.
- Memastikan kebutuhan pengguna terpenuhi.
- Menyediakan proses pengembangan yang terstruktur.
- Memudahkan pengujian dan pemeliharaan.

Tahapan Umum SDLC

Planning – menentukan tujuan dan ruang lingkup.

Requirements Analysis – mengidentifikasi kebutuhan pengguna.

Design – merancang arsitektur dan komponen sistem.

Implementation – melakukan pemrograman.

Testing – memastikan sistem bekerja sesuai kebutuhan.

Deployment – menerapkan sistem kepada pengguna.

Maintenance – memperbaiki dan mengembangkan sistem.

Inti: SDLC bukan hanya metode pemrograman, tetapi pendekatan untuk mengelola keseluruhan siklus hidup perangkat lunak.

Model Waterfall – Linear Sequential Model

Pengertian: model pengembangan perangkat lunak yang menggunakan tahapan berurutan. Setiap tahap umumnya diselesaikan sebelum tahap berikutnya dimulai.

Alur: Requirements → Design → Implementation → Testing → Deployment → Maintenance

Karakteristik:

- Proses bersifat linear.
- Dokumentasi relatif kuat.
- Perubahan kebutuhan setelah tahap awal relatif sulit.
- Setiap fase memiliki output yang jelas.

Waterfall: Kelebihan, Kekurangan & Kesesuaian

Kelebihan:

- Sederhana dan mudah dipahami.
- Tahapan dan tanggung jawab jelas.
- Dokumentasi terstruktur.
- Mudah digunakan untuk proyek dengan kebutuhan stabil.

Kekurangan:

- Kurang fleksibel terhadap perubahan.
DevSecOps
- Pengguna biasanya baru melihat produk secara nyata pada tahap akhir.
- Risiko kesalahan kebutuhan dapat terlambat diketahui.

Cocok digunakan ketika: kebutuhan sudah jelas; teknologi matang; regulasi dan dokumentasi penting; perubahan kebutuhan rendah.

Contoh: sistem administrasi dengan prosedur dan kebutuhan yang sudah ditetapkan sejak awal.

Contoh Model Waterfall

Cocok ketika kebutuhan relatif stabil dan tahapan perlu didokumentasikan dengan jelas.

Kasus: sistem pengajuan cuti pegawai dengan aturan, formulir, alur persetujuan, dan laporan yang sudah ditetapkan.

VISUALISASI ALUR



CATATAN PENTING

Contoh output: dokumen kebutuhan disetujui lebih dahulu; setelah itu desain dan implementasi mengikuti baseline yang telah ditetapkan.

Diskusi: bagian mana dari contoh ini yang paling menentukan keberhasilan proyek?

Ilustrasi konseptual untuk pembelajaran; struktur nyata dapat berbeda sesuai kebutuhan proyek.

MODEL PROTOTYPING

Memahami kebutuhan melalui model awal dan umpan balik pengguna.

Model Prototyping

Pengertian: pendekatan pengembangan yang membuat model awal atau prototype sistem untuk membantu pengguna memahami dan memberikan umpan balik terhadap kebutuhan sistem.

Alur: Kebutuhan Awal → Prototype → Evaluasi Pengguna → Perbaikan → Prototype/Sistem

Jenis prototype:

- Throwaway prototype – dibuat untuk memahami kebutuhan kemudian dibuang.
- Evolutionary prototype – dikembangkan secara bertahap menjadi sistem final.

Prototyping: Kelebihan, Kekurangan & Contoh

Kelebihan:

- Pengguna lebih mudah memahami rancangan.
- Kebutuhan yang belum jelas dapat dieksplorasi.
- Feedback diperoleh lebih awal.
- Mengurangi risiko salah memahami kebutuhan.

Kekurangan:

- Pengguna dapat menganggap prototype sebagai produk final.
- Pengembang dapat terlalu fokus pada tampilan.
- Dokumentasi dapat terabaikan.
- Perubahan berulang dapat meningkatkan biaya.

Cocok untuk: sistem dengan kebutuhan belum jelas; aplikasi berorientasi pengguna; sistem dengan antarmuka kompleks.

Contoh: aplikasi mobile kampus yang membutuhkan banyak masukan mahasiswa dan dosen mengenai desain antarmuka.

Contoh Model Prototyping

Cocok ketika kebutuhan belum sepenuhnya jelas dan pengguna perlu mencoba rancangan awal.

Kasus: aplikasi mobile kampus. Mahasiswa mencoba mock-up halaman KRS, lalu memberi masukan tentang navigasi, jadwal, dan tampilan.

VISUALISASI ALUR



CATATAN PENTING

Prototype dapat dibuang setelah kebutuhan jelas (throwaway) atau dikembangkan menjadi produk (evolutionary).

Diskusi: bagian mana dari contoh ini yang paling menentukan keberhasilan proyek?

Ilustrasi konseptual untuk pembelajaran; struktur nyata dapat berbeda sesuai kebutuhan proyek.

INCREMENTAL & ITERATIVE

Membangun sistem secara bertahap dan menyempurnakannya melalui pengulangan.

Model Incremental

Incremental berarti sistem dikembangkan dan diberikan kepada pengguna dalam beberapa bagian atau increment.

Contoh:

- Increment 1 → Login
- Increment 2 → Data Mahasiswa
- Increment 3 → KRS
- Increment 4 → Pembayaran

Setiap increment menambahkan kemampuan baru.

Kelebihan: produk dapat digunakan lebih awal; feedback diperoleh secara bertahap; risiko dapat dibagi menjadi beberapa bagian; prioritas fitur dapat disesuaikan.

Model Iterative

Iterative berarti sistem dikembangkan melalui pengulangan. Setiap iterasi menghasilkan versi yang semakin disempurnakan.

Version 1 → Feedback → Version 2 → Feedback → Version 3

Perbedaan sederhana:

- Incremental: menambahkan fitur baru; fokus pada pertambahan fungsi; produk berkembang secara bertahap.
- Iterative: menyempurnakan fitur; fokus pada perbaikan; produk mengalami penyempurnaan berulang.

Dalam praktik: Incremental dan iterative sering digunakan bersamaan.

Contoh Model Iterative

Sistem diperbaiki melalui siklus berulang berdasarkan hasil pengujian dan umpan balik.

Kasus: fitur pencarian mata kuliah. Versi awal dibuat, lalu disempurnakan agar pencarian lebih cepat dan filter lebih mudah digunakan.

VISUALISASI ALUR



Kualitas disempurnakan



CATATAN PENTING

Fokus utama: penyempurnaan kualitas atau perilaku solusi pada setiap iterasi. Dalam praktik, iterative dapat digabung dengan incremental.

Diskusi: bagian mana dari contoh ini yang paling menentukan keberhasilan proyek?

Ilustrasi konseptual untuk pembelajaran; struktur nyata dapat berbeda sesuai kebutuhan proyek.

MODEL SPIRAL

Risk-Driven Model — menggabungkan pendekatan iteratif dengan analisis risiko.

Model Spiral – Risk-Driven Model

Pengertian: model pengembangan yang menggabungkan pendekatan iteratif dengan analisis risiko secara sistematis.

Empat aktivitas utama:

1. Planning – menentukan tujuan dan kebutuhan.
2. Risk Analysis – mengidentifikasi dan menganalisis risiko.
3. Engineering – mengembangkan dan menguji solusi.
4. Evaluation – mengevaluasi hasil bersama stakeholder.

Kemudian siklus berlanjut ke putaran berikutnya.

Fokus utama: Risk Management.

Spiral: Kelebihan, Kekurangan & Contoh

Kelebihan: risiko diidentifikasi sejak awal; cocok untuk proyek besar dan kompleks; mendukung perubahan kebutuhan; feedback dapat diperoleh pada setiap siklus.

Kekurangan: relatif kompleks; membutuhkan keahlian analisis risiko; biaya pengelolaan lebih tinggi; tidak selalu cocok untuk proyek kecil.

Contoh: sistem perbankan skala besar; sistem kesehatan; sistem pertahanan; sistem dengan risiko keamanan tinggi.

Contoh Model Spiral

Pendekatan berbasis risiko; setiap putaran merencanakan, menganalisis risiko, membangun, lalu mengevaluasi.

Kasus: sistem pembayaran kampus yang harus melindungi transaksi, menjaga ketersediaan, dan terhubung dengan penyedia pembayaran.

VISUALISASI ALUR

1. Planning	2. Risk analysis	3. Engineering	4. Evaluation
Tujuan & batas	Fraud, privasi	Prototype & uji	Review stakeholder

Siklus berulang (ilustrasi)



CATATAN PENTING

Risiko prioritas menentukan pekerjaan pada putaran berikutnya. Cocok untuk proyek besar/kompleks, tetapi butuh keahlian dan biaya pengelolaan lebih tinggi.

Diskusi: bagian mana dari contoh ini yang paling menentukan keberhasilan proyek?

Ilustrasi konseptual untuk pembelajaran; struktur nyata dapat berbeda sesuai kebutuhan proyek.

MODEL RAD

Rapid Application Development — kecepatan, prototyping, komponen dan keterlibatan pengguna.

Model RAD – Rapid Application Development

Pengertian: pendekatan pengembangan yang menekankan kecepatan pembangunan aplikasi melalui prototyping, penggunaan komponen, dan keterlibatan pengguna secara intensif.

Prinsip utama: pengembangan cepat; prototyping; user involvement; reusable components; iterasi singkat.

Tahapan umum: Requirements Planning → User Design → Construction → Cutover

RAD: Kelebihan, Kekurangan & Contoh

Kelebihan: waktu pengembangan relatif cepat; feedback pengguna diperoleh dengan cepat; cocok untuk aplikasi bisnis; menggunakan kembali komponen yang tersedia.

Kekurangan: membutuhkan pengguna yang aktif; kurang cocok untuk sistem yang sangat kompleks; membutuhkan tenaga pengembang yang terampil; tidak semua proyek dapat dikembangkan dengan cepat.

Contoh: aplikasi internal perusahaan yang membutuhkan sistem informasi sederhana dalam waktu relatif singkat.

Contoh Model RAD

Rapid Application Development menekankan prototyping cepat, keterlibatan pengguna, dan komponen siap pakai.

Kasus: aplikasi inventaris internal untuk unit kampus, dibangun cepat dengan template UI, database, dan masukan langsung staf.

VISUALISASI ALUR



CATATAN PENTING

RAD efektif jika ruang lingkup dapat dibagi, pengguna aktif memberi feedback, dan tim berpengalaman. Tidak semua sistem dapat dipercepat dengan cara ini.

Diskusi: bagian mana dari contoh ini yang paling menentukan keberhasilan proyek?

Ilustrasi konseptual untuk pembelajaran; struktur nyata dapat berbeda sesuai kebutuhan proyek.

MODEL V-SHAPE

Verification and Validation — menghubungkan tahapan pengembangan dengan pengujian.

Model V-Shape – Verification and Validation

Pengertian: model pengembangan yang menghubungkan setiap tahap pengembangan dengan tahap pengujian yang sesuai.

Model ini berbentuk seperti huruf V.

Sisi kiri – Verification: Requirements → System Design → Architecture Design → Module Design → Coding

Sisi kanan – Validation: Unit Testing → Integration Testing → System Testing → Acceptance Testing

V-Model: Konsep, Kelebihan, Kekurangan

Verification: “Are we building the product right?”

Validation: “Are we building the right product?”

Kelebihan: pengujian direncanakan sejak awal; hubungan kebutuhan dan pengujian jelas; cocok untuk sistem yang membutuhkan kualitas dan dokumentasi tinggi.

Kekurangan: kurang fleksibel terhadap perubahan; perubahan kebutuhan dapat mahal; kurang cocok untuk proyek dengan kebutuhan yang belum jelas.

Contoh: sistem medis; sistem otomotif; sistem penerbangan; sistem yang memiliki tuntutan keselamatan tinggi.

Contoh Model V-Shape

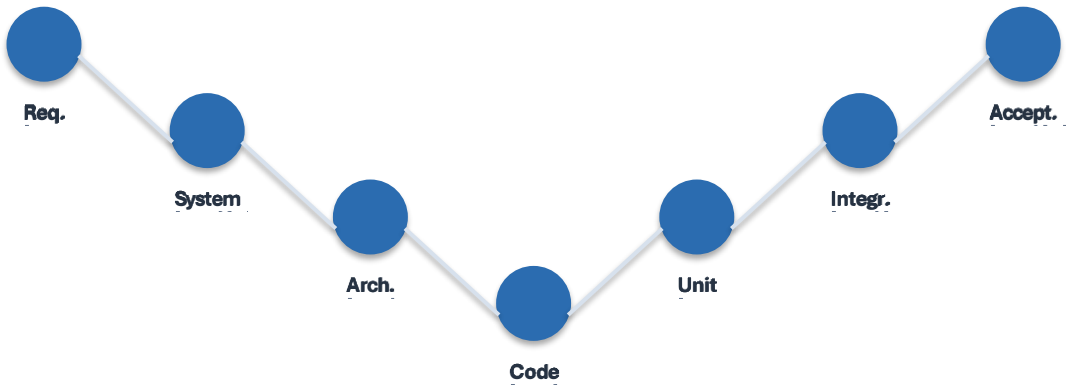
Setiap tahap spesifikasi/perancangan di sisi kiri dipasangkan dengan tingkat pengujian yang sesuai di sisi kanan.

Kasus: sistem presensi laboratorium. Kebutuhan akses, integrasi perangkat, dan fungsi modul direncanakan bersama pengujiannya.

VISUALISASI ALUR

Requirements	System design	Architecture	Module design
↔ Acceptance test	↔ System test	↔ Integration test	↔ Unit test

Bentuk V: desain ↔ pengujian



CATATAN PENTING

Pengujian direncanakan sejak awal. Coding berada di dasar huruf V; pasangan pengujian di sisi kanan memeriksa hasil pada tingkat yang sesuai.

Diskusi: bagian mana dari contoh ini yang paling menentukan keberhasilan proyek?

Ilustrasi konseptual untuk pembelajaran; struktur nyata dapat berbeda sesuai kebutuhan proyek.

COMPONENT-BASED DEVELOPMENT

Membangun software dengan memanfaatkan komponen yang dapat digunakan kembali.

Component-Based Development

Pengertian: pendekatan pengembangan perangkat lunak dengan menggunakan komponen yang sudah tersedia dan dapat digunakan kembali.

Konsep utama: Component → Reuse → Integration → Application

Contoh komponen: Authentication module; Payment gateway; Database component; Reporting module; UI component; API.

Component-Based Development: Proses & Evaluasi

Proses: 1) Identifikasi kebutuhan; 2) Mencari komponen yang tersedia; 3) Mengevaluasi komponen; 4) Melakukan adaptasi; 5) Mengintegrasikan komponen; 6) Melakukan pengujian.

Kelebihan: mengurangi waktu pengembangan; mengurangi pekerjaan pemrograman berulang; meningkatkan penggunaan kembali software; memanfaatkan komponen yang telah diuji.

Kekurangan: ketergantungan terhadap komponen pihak ketiga; masalah kompatibilitas; risiko keamanan komponen; versi komponen dapat berubah.

Component-Based Development: Contoh

Pengembang menggunakan:

- library autentikasi
- payment API
- framework UI
- database driver
- cloud services

daripada membangun semuanya dari nol.

Contoh Component-Based Development

Aplikasi dibangun dengan mengintegrasikan komponen yang sudah tersedia dan dievaluasi kelayakannya.

Kasus: toko daring kampus menggunakan framework UI, layanan autentikasi, payment gateway, dan library laporan yang tersedia.

VISUALISASI ALUR



CATATAN PENTING

Reuse mengurangi pekerjaan berulang, tetapi tim tetap harus memeriksa kompatibilitas, lisensi, dukungan versi, dan risiko keamanan.

Diskusi: bagian mana dari contoh ini yang paling menentukan keberhasilan proyek?

Ilustrasi konseptual untuk pembelajaran; struktur nyata dapat berbeda sesuai kebutuhan proyek.

PEMILIHAN MODEL PROSES

Tidak ada satu model SDLC yang selalu paling baik.

Pemilihan Model Proses yang Tepat

Model harus dipilih berdasarkan karakteristik proyek.

Faktor yang perlu dipertimbangkan:

1. Stabilitas kebutuhan: Stabil → Waterfall/V-Model; Berubah → Iterative/Agile
2. Tingkat risiko: Risiko tinggi → Spiral
3. Kecepatan pengembangan: Sangat membutuhkan kecepatan → RAD
4. Keterlibatan pengguna: Tinggi → Prototyping/Iterative
5. Kompleksitas sistem: Kompleks → Spiral/Iterative
6. Kebutuhan kualitas dan regulasi: Tinggi → V-Model/Waterfall dengan kontrol yang kuat
7. Ketersediaan komponen: Banyak komponen siap pakai → Component-Based Development

Panduan Pemilihan Model

Kebutuhan sangat jelas → Waterfall

Kebutuhan belum jelas → Prototyping

Fitur dikembangkan bertahap → Incremental

Membutuhkan penyempurnaan berulang → Iterative

Risiko sangat tinggi → Spiral

Membutuhkan pengembangan cepat → RAD

Pengujian sangat ketat → V-Model

Banyak komponen tersedia → Component-Based

EVOLUSI MODEL PROSES KE ARAH MODERN

Dari model linear menuju proses adaptif, otomatis, terintegrasi dan berbantuan AI.

Evolusi Model Proses

Waterfall

↓

Prototyping

↓

Incremental / Iterative

↓

Spiral / RAD

↓

Agile

↓

DevOps

↓

DevSecOps / Cloud-Native / AI-Assisted Development

Mengapa Terjadi Evolusi?

Perangkat lunak modern membutuhkan:

- Pengembangan lebih cepat.
- Feedback pengguna lebih sering.
- Continuous Integration.
- Continuous Delivery.
- Otomasi testing.
- Cloud computing.
- Keamanan sejak awal.
- Integrasi lintas tim.
- Adaptasi terhadap perubahan.
- Pemanfaatan AI dalam pengembangan software.

AGILE

Pendekatan adaptif yang menekankan iterasi singkat, kolaborasi dan feedback.

Model Modern: Agile

Pengertian: Agile merupakan pendekatan pengembangan yang menekankan iterasi singkat, kolaborasi, feedback pengguna, dan kemampuan beradaptasi terhadap perubahan.

Prinsip utama: Customer collaboration; Working software; Responding to change; Continuous feedback; Cross-functional team.

Contoh framework: Scrum; Kanban; Extreme Programming (XP).

Contoh Siklus Scrum

Product Backlog → Sprint Planning → Sprint → Review → Retrospective → Sprint berikutnya

Product Backlog: daftar kebutuhan atau pekerjaan yang diprioritaskan.

Sprint: periode pengembangan dengan tujuan yang jelas.

Review: mengevaluasi hasil sprint.

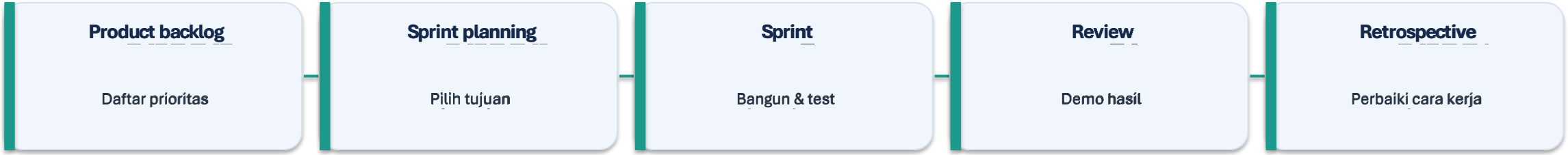
Retrospective: merefleksikan proses dan menentukan perbaikan.

Contoh Agile (Scrum)

Tim mengembangkan perangkat lunak dalam sprint singkat dengan prioritas dan feedback yang terus diperbarui.

Kasus: tim mahasiswa membuat aplikasi peminjaman ruang. Sprint pertama untuk login, sprint berikutnya untuk jadwal dan pemesanan.

VISUALISASI ALUR



CATATAN PENTING

Agile adalah pendekatan; Scrum merupakan salah satu framework-nya. Perubahan prioritas dikelola melalui backlog dan siklus kerja.

Diskusi: bagian mana dari contoh ini yang paling menentukan keberhasilan proyek?

Ilustrasi konseptual untuk pembelajaran; struktur nyata dapat berbeda sesuai kebutuhan proyek.

DEVOPS

Development + Operations — mempercepat delivery dan feedback melalui kolaborasi dan otomasi.

DevOps sebagai Evolusi SDLC

Pengertian: DevOps mengintegrasikan aktivitas Development dan Operations agar software dapat dikembangkan, diuji, dirilis, dan dipelihara secara lebih berkelanjutan.

Siklus: Plan → Code → Build → Test → Release → Deploy → Operate → Monitor → kembali ke Plan.

Tujuan: mempercepat delivery; mengurangi kesalahan manual; meningkatkan kolaborasi; mendukung continuous delivery; mempercepat feedback.

Contoh DevOps dan DevSecOps

Otomatisasi dan kolaborasi membantu aplikasi dirilis secara berkelanjutan, dipantau, dan diperbaiki.

Kasus: portal akademik diperbarui berkala. Setiap perubahan kode diuji otomatis sebelum dirilis ke lingkungan produksi.

VISUALISASI ALUR



CATATAN PENTING

DevOps menekankan kolaborasi Development–Operations. DevSecOps mengintegrasikan keamanan ke seluruh siklus; keduanya adalah pendekatan praktik, bukan sekadar urutan fase.

Diskusi: bagian mana dari contoh ini yang paling menentukan keberhasilan proyek?

Ilustrasi konseptual untuk pembelajaran; struktur nyata dapat berbeda sesuai kebutuhan proyek.

AI DALAM SDLC

Artificial Intelligence semakin terintegrasi ke berbagai aktivitas pengembangan software.

AI dalam Model Proses Perangkat Lunak

Perkembangan terbaru membawa Artificial Intelligence (AI) ke berbagai aktivitas SDLC.

AI dapat membantu:

- Analisis kebutuhan.
- Pembuatan dokumentasi.
- Code generation.
- Code review.
- Software testing.
- Bug detection.
- Test case generation.
- Refactoring.
- Monitoring.
- Analisis log.

Alur AI-Assisted Software Development

Human Requirement → AI-Assisted Analysis → Development → Automated Testing → CI/CD → Deployment → Monitoring → Feedback

AI membantu proses pengembangan, tetapi keputusan teknis, validasi, keamanan, dan tanggung jawab akhir tetap berada pada manusia.

Ringkasan Perbandingan Model

Waterfall — Fokus: Tahapan berurutan | Fleksibilitas: Rendah | Risiko: Sedang | Kecepatan: Sedang

Prototyping — Fokus: Klarifikasi kebutuhan | Fleksibilitas: Tinggi | Risiko: Sedang | Kecepatan: Sedang

Incremental — Fokus: Penambahan fitur | Fleksibilitas: Tinggi | Risiko: Sedang | Kecepatan: Tinggi

Iterative — Fokus: Penyempurnaan | Fleksibilitas: Tinggi | Risiko: Sedang | Kecepatan: Tinggi

Spiral — Fokus: Manajemen risiko | Fleksibilitas: Tinggi | Risiko: Tinggi | Kecepatan: Sedang

RAD — Fokus: Pengembangan cepat | Fleksibilitas: Tinggi | Risiko: Sedang | Kecepatan: Sangat tinggi

V-Model — Fokus: Verification & Validation | Fleksibilitas: Rendah | Risiko: Rendah–Sedang | Kecepatan: Sedang

Component-Based — Fokus: Reuse komponen | Fleksibilitas: Tinggi | Risiko: Bergantung komponen | Kecepatan: Tinggi

Agile — Fokus: Adaptasi & feedback | Fleksibilitas: Sangat tinggi | Risiko: Dikelola iteratif | Kecepatan: Tinggi

DevOps — Fokus: Delivery berkelanjutan | Fleksibilitas: Sangat tinggi | Risiko: Dikelola otomatisasi | Kecepatan: Sangat tinggi

Kesimpulan

1. SDLC merupakan kerangka untuk mengelola seluruh siklus hidup perangkat lunak.
2. Tidak ada satu model yang cocok untuk semua proyek. Pemilihan model bergantung pada kebutuhan, risiko, kompleksitas, waktu, dan karakteristik pengguna.
3. Model tradisional seperti Waterfall cocok ketika kebutuhan relatif stabil.
4. Model adaptif seperti Iterative, Incremental, Agile, dan RAD lebih sesuai ketika perubahan dan feedback menjadi bagian penting dari proyek.
5. Model modern bergerak menuju otomatisasi, integrasi, continuous delivery, cloud, security, dan AI-assisted software development.

Pertanyaan Diskusi Mahasiswa

1. Mengapa Waterfall kurang cocok untuk kebutuhan yang sering berubah?
2. Apa perbedaan utama Incremental dan Iterative?
3. Mengapa Spiral disebut Risk-Driven Model?
4. Dalam kondisi apa RAD lebih tepat digunakan?
5. Mengapa pengujian pada V-Model direncanakan sejak awal?
6. Apa keuntungan menggunakan Component-Based Development?
7. Faktor apa saja yang harus dipertimbangkan ketika memilih model SDLC?
8. Mengapa Agile muncul sebagai respons terhadap perubahan kebutuhan software?
9. Apa hubungan Agile dengan DevOps?
10. Bagaimana AI dapat mengubah aktivitas SDLC di masa depan?

Pesan Utama

“Model proses bukan sekadar urutan tahapan pengembangan, tetapi strategi untuk mengelola kebutuhan, risiko, kualitas, waktu, biaya, dan perubahan dalam pembangunan perangkat lunak.”

Mata Kuliah: Rekayasa Perangkat Lunak

Topik: Model Proses Perangkat Lunak (SDLC)

Dosen: Manorang Sihotang